

Ecosistema de Automatización IA Autónomo para Negocios

Triage inteligente de tickets de soporte con validación humana (HITL)

Autor: Armando Ismael

Caso de uso: Atención al cliente — clasificación y respuesta asistida por IA

Stack: Make · Airtable · OpenAI (GPT-4o-mini) · Slack

Documento: Documentación Técnica (Criterios 1 a 4)

Ecosistema de Automatización IA · Entrega Final Coder 1

Documentación Técnica del Ecosistema

Este documento describe la arquitectura, las estructuras de datos, la estrategia de modelos de IA y la malla de seguridad y resiliencia del sistema. El sistema está funcionando en vivo sobre Make (orquestador), Airtable (base de datos/memoria), OpenAI (motor de razonamiento) y Slack (canal de salida + punto de validación humana).

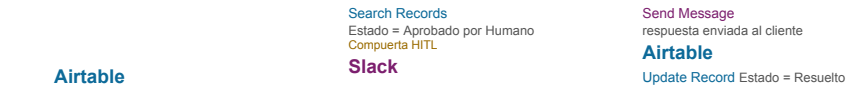
Criterio 1 Mapa de Arquitectura del Sistema

El ecosistema resuelve un proceso de negocio de extremo a extremo: un ticket entra a la base de datos, la IA lo interpreta y clasifica, se guarda el resultado, y el sistema **se detiene ante la acción crítica** (responder al cliente) hasta que un humano aprueba. La publicación y el cierre ocurren en un segundo flujo condicional.

Flujo 1 — Generación + Pausa HITL (escenario "Final - Triage IA")



Flujo 2 — Publicación Condicional (escenario "Final - Publicacion HITL")



Componentes de la arquitectura

Trigger de entrada	Airtable · Watch Records ({Estado} = 'Pendiente')	Dispara el flujo solo ante tickets nuevos (evita bucles y consumo innecesario)
Nodo de IA	OpenAI · GPT-4o-mini (salida JSON)	Clasifica categoría, urgencia y sentimiento; redacta respuesta
Router / Compuerta	Filtro "Compuerta HITL" (Estado = Aprobado por Humano)	Solo deja pasar tickets aprobados por un humano
APIs integradas	Slack (salida) · Airtable (memoria)	Notificación/entrega y persistencia de estado
Destino de datos	Airtable (Tickets, Errores_Log)	Registro único de verdad del sistema
Ruta de error	Error Handler → Airtable Create + Resume	Registra el fallo y continúa sin romperse

Criterio 2 Manual Operativo de Estructuras de Datos

El "cerebro" del sistema es una base relacional en **Airtable** ("Triage de Tickets con IA") con **tres tablas vinculadas**. La sincronización entre estados la gestiona íntegramente el flujo de automatización (no hay edición manual de datos intermedios).

Tabla 1 — Tickets (tabla principal)

Ecosistema de Automatización IA · Entrega Final Coder 2

Remitente	Texto	Email/nombre del cliente
Asunto / Cuerpo	Texto	Contenido del ticket (entrada de la IA)
Categoría	Selección	Soporte Técnico · Recursos Humanos · Administrativo · General...
Urgencia	Selección	Alta · Media · Baja
Sentimiento	Selección	Frustración · Neutral · Preocupación · Esperanza
Respuesta_Sugerida	Texto largo	Borrador redactado por la IA
Estado	Selección	Campo de estado del ciclo (ver máquina de estados)

Aprobado_Por	Texto	Quién validó (registro del HITL)
Agente_Asignado	Vínculo → Agentes	Relación entre tablas (evita datos aislados)
Thread_ID_Slack	Texto	Hilo de Slack asociado

Tabla 2 — Agentes · Tabla 3 — Errores_Log

Agentes: catálogo de responsables, vinculada a Tickets por el campo **Agente_Asignado** . **Errores_Log:** registro de fallos con campos **Escenario** , **Tipo_Error** (API Caída, Timeout, Dato Faltante, Validación...), **Payload** y **Estado_Error** (Nuevo, Revisado, Resuelto).

Máquina de estados (campos de estado obligatorios):
Pendiente → **Esperando Aprobación** → **Aprobado por Humano** → **Resuelto** | los fallos derivan a **Errores_Log** .

Esquemas de transferencia de datos (JSON)

Salida estructurada del nodo de IA (contrato JSON que consume el resto del flujo):

```
{
  "categoria": "Soporte Técnico | Recursos Humanos | Administrativo",
  "urgencia": "Alta | Media | Baja",
  "sentimiento": "Frustración | Neutral | Preocupación | Esperanza",
  "respuesta_sugerida": "texto de respuesta al cliente"
}
```

Payload de actualización a Airtable (Flujo 1):

```
{
  "id": "{{ticket_id}}",
  "record": {
    "Estado": "Esperando Aprobación",
    "Categoria": "{{ia.categoria}}",
    "Urgencia": "{{ia.urgencia}}",
    "Sentimiento": "{{ia.sentimiento}}",
    "Respuesta_Sugerida": "{{ia.respuesta_sugerida}}"
  }
}
```

Registro de error hacia Errores_Log (ruta de contingencia):

```
{
  "Escenario": "Final - Triage IA (generacion HITL)",
  "Tipo_Error": "API Caída",
  "Estado_Error": "Nuevo",
  "Payload": "Ticket ID + Asunto + Cuerpo del ticket afectado"
}
```

Criterio 3 Estrategia de Selección de Herramientas y Modelos

La selección de modelos se rige por un principio de **rentabilidad**: usar el modelo más económico que resuelva bien cada tarea, reservando los modelos potentes para el trabajo que realmente los necesita.

Ecosistema de Automatización IA - Entrega Final Coder 3

Mecánica y en vivo (clasificar un ticket, redactar respuesta corta)	OpenAI GPT-4o-mini	Rápido y barato; ideal para respuestas inmediatas de bajo costo. <i>Es el que usa el sistema en producción.</i>
Lectura densa / razonamiento complejo (analizar documentos largos, contratos)	Anthropic Claude	Ventana de contexto de 200k tokens y razonamiento paso a paso; menor tasa de alucinación en textos extensos
Procesos masivos no urgentes (clasificar miles de tickets históricos)	Message Batches API + Prompt Caching	50% de ahorro procesando de forma asíncrona (hasta 24 h); el caching reduce el costo de la instrucción repetida

Matriz de decisión (regla operativa):

.

¿La respuesta se necesita **ya** y la tarea es simple? → GPT-4o-mini.

- ¿Hay que **leer y razonar** sobre mucho texto? → Claude.
- ¿La tarea **puede esperar** a la noche y es de gran volumen? → Batches (mitad de precio).

Optimización de costos adicional: Límite de **Max Tokens** (500) para acotar el gasto por llamada, y prompt con variables dinámicas (sin texto hardcodeado).

Criterio 4 Malla de Seguridad, Privacidad y Resiliencia

Minimización de datos y privacidad

- Al modelo de IA solo se le envían los campos necesarios (**Asunto** y **Cuerpo**); no se exponen datos sensibles adicionales. • Las credenciales (API Keys de OpenAI, tokens de Slack y Airtable) viven en las **conexiones de Make**, nunca hardcodeadas en el flujo ni visibles en el blueprint.
- Variables dinámicas en todos los nodos: sin datos fijos incrustados.

Rutas de contingencia (Error Handlers en Make)

El módulo de IA cuenta con un manejador de errores que, ante un fallo de la API:

- **Registra** el incidente en la tabla **Errores_Log** (escenario, tipo de error, payload, estado).
- **Continúa** con la directiva **Resume** , entregando un valor sustituto ("requiere revisión manual") para que el flujo no se rompa y el ticket no se pierda.
- Alternativa disponible: directiva **Break** con reintentos automáticos (3 intentos) para fallos transitorios de red.

Directivas de error de Make consideradas: Rollback (deshace y detiene), Ignore (omite el error), Break (reintenta) y Resume (continúa con sustituto). El sistema usa **Resume** por su degradación elegante.

Puntos de validación humana (Human-in-the-Loop)

El sistema inserta un "semáforo" humano **antes de la acción crítica** (responder al cliente): la IA nunca contacta al exterior por su cuenta. El Flujo 1 deja el ticket en **Esperando Aprobación** y avisa por Slack; solo cuando un humano cambia el estado a **Aprobado por Humano** , el Flujo 2 publica la respuesta. Esto mitiga el riesgo legal y las alucinaciones de la IA antes de que lleguen al cliente.

Checklist de seguridad verificado:

- - ✓ Filtro en el trigger para evitar bucles infinitos (solo **Estado = Pendiente**). •
 - ✓ Comparación de tipos correcta en los filtros (estado exacto por texto). •
 - ✓ Prompt de IA dinámico con variables del sistema (**{{Asunto}}** , **{{Cuerpo}}**). •
 - ✓ Registro de errores probado en vivo (camino infeliz → fila en Errores_Log).

